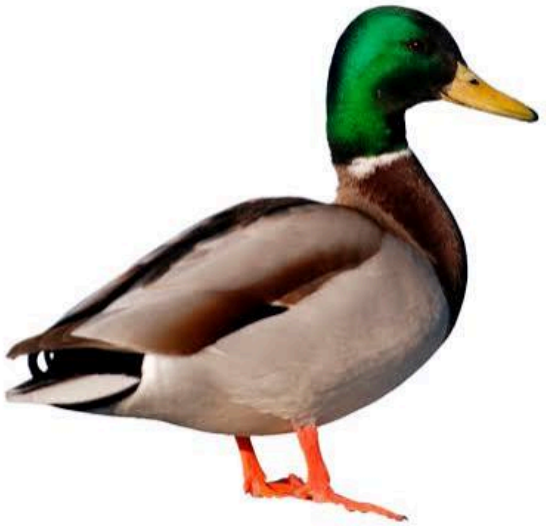
Ciência de Dados Aplicada ao Direito I

Aula 14

Dos registros aos documentos



Prof. Bruno da Cunha de Oliveira - 2026/2

Consultar pelo nome do campo

A posição ainda responde à pergunta?

```
caso = ["Marina Alves", True]
print(caso[0])

caso = [True, "Marina Alves"]
print(caso[0])
```

Os casos chegam ao gabinete com o nome de quem responde e se houve vantagem ilícita. A ordem dos campos mudou. Como consultar o nome sem depender da posição zero?

A posição ainda responde à pergunta?

RESPOSTA

```
caso = {"investigado": "Marina Alves", "obteve_vantagem": True}
print(caso["investigado"])
```

```
Marina Alves
```

O dicionário associa uma chave a cada valor. A consulta pede o campo pelo nome, não pela posição.

Chave e valor

```
caso = {
    "investigado": "Marina Alves",
    "obteve_vantagem": True,
}
print(caso["obteve_vantagem"])
print(len(caso))
```

```
True
2
```

Chaves delimitam o dicionário; cada par tem `chave: valor`. Os colchetes consultam uma chave. `len()` conta os pares.

Registrar o prejuízo alheio

A análise do caso de Marina apurou que houve prejuízo alheio, e o registro ainda não tem esse campo.

```
caso = {"investigado": "Marina Alves", "obteve_vantagem": False}
caso["obteve_vantagem"] = True
caso["houve_prejuizo"] = True
```

Na lista, pedir uma posição que não existe causa erro. O que acontece ao atribuir a uma chave que não existe?

Registrar o prejuízo alheio

RESPOSTA

```
print(caso["obteve_vantagem"])
print(caso["houve_prejuizo"])
```

```
True
True
```

A chave existente recebe outro valor; a chave nova cria um par. Não precisamos de `.append()` para acrescentar um campo.

Que meio foi empregado?

```
caso = {"investigado": "Marina Alves", "obteve_vantagem": True}
print(caso["meio_fraudulento"])
```

```
KeyError: 'meio_fraudulento'
```

A consulta parou porque essa chave não existe. Como conferir antes de pedir o valor?

Que meio foi empregado?

RESPOSTA

```
if "meio_fraudulento" in caso:
    print(caso["meio_fraudulento"])
else:
    print("Meio fraudulento não apurado")
```

No dicionário, `in` pergunta pelas chaves, não pelos valores. `"Marina Alves" in caso` é `False`, embora seja o valor de `"investigado"`.

Aplicar a regra a todos os registros

Uma volta por caso

```
casos = [{"investigado": "Marina Alves", "configura": True},
         {"investigado": "Otávio Bastos", "configura": False}]
for caso in casos:
    print(caso["investigado"])
    print(caso["configura"])
```

O `for` continua percorrendo uma lista. Cada item agora é um dicionário; dentro da volta, consultamos as chaves.

Uma conclusão para cada caso

```
casos = [{"investigado": "Marina Alves", "configura": True},
         {"investigado": "Otávio Bastos", "configura": False}]
conclusoes = []
for caso in casos:
    if caso["configura"]:
        conclusao = caso["investigado"] + ": denúncia"
    else:
        conclusao = caso["investigado"] + ": arquivamento"
    conclusoes.append(conclusao)
print(conclusoes)
```

```
['Marina Alves: denúncia', 'Otávio Bastos: arquivamento']
```

Mostrar os campos não decide nada: o gabinete precisa de uma conclusão por caso. A condição escolhe o encaminhamento, e a inclusão ocorre qualquer que seja ele.

Uma conclusão para cada caso

Animação no navegador: `animacao-loop.html`, cenário `dicionarios`.

Separar os casos que vão a arquivamento

```
casos = [{"investigado": "Marina Alves", "configura": True},
         {"investigado": "Otávio Bastos", "configura": False}]
arquivamentos = []
for caso in casos:
    if not caso["configura"]:
        arquivamentos.append(caso["investigado"])
print(arquivamentos)
print(len(arquivamentos))
```

Queremos os nomes de quem não será denunciado. O que mudou na quantidade de itens? Onde está o `.append()`?

Separar os casos que vão a arquivamento

RESPOSTA

```
['Otávio Bastos']
1
```

O `.append()` está dentro do `if`, e o `not` inverte a condição. A lista registra somente quem satisfez a condição. Selecionar os casos é diferente de concluir sobre todos eles.

Separar os casos que vão a arquivamento

Animação no navegador: `animacao-loop.html`, cenário `pendentes`.

No PrairieLearn: exercício 3

Insper

Preencher documentos

Dois despachos de arquivamento

O gabinete despacha cada caso arquivado. Estes são os de Otávio e de Helena.

```
DESPACHO
Autos: 0000456-76.2024.8.99.0001
Investigado: Otávio Bastos
Arquive-se.
```

```
DESPACHO
Autos: 0000789-25.2024.8.99.0002
Investigado: Helena Braga
Arquive-se.
```

O que muda de um despacho para o outro? O que seria preciso guardar para escrever o terceiro?

Uma minuta com campos variáveis

RESPOSTA

Mudam dois campos. Guardamos o texto uma vez só, com um marcador no lugar de cada campo que varia.

```
modelo = """DESPACHO
Autos: <AUTOS>
Investigado: <NOME>
Arquive-se."""
print(modelo)
```

As três aspas guardam as quebras de linha. Os marcadores são texto comum: Python ainda não sabe o que deve colocar neles.

Substituir o nome

```
modelo = "Investigado: <NOME>"
modelo.replace("<NOME>", "Otávio Bastos")
print(modelo)
```

A chamada troca o marcador pelo nome. Qual texto o `print()` vai mostrar?

Substituir o nome

RESPOSTA

```
Investigado: <NOME>
```

`.replace()` devolve outro texto não altera o original. Precisamos guardar o resultado.

```
peca = modelo.replace("<NOME>", "Otávio Bastos")
print(peca)
```

```
Investigado: Otávio Bastos
```

A vantagem apurada é um número

```
vantagem = 4500
modelo = "Vantagem apurada: R$ <VALOR>"
peca = modelo.replace("<VALOR>", vantagem)
```

```
TypeError: replace() argument 2 must be str, not int
```

O segundo argumento de `.replace()` precisa ser texto. O valor apurado é um número; como inseri-lo sem trocar o dado à mão?

str(): representar o número como texto

```
vantagem = 4500
modelo = "Vantagem apurada: R$ <VALOR>"
peca = modelo.replace("<VALOR>", str(vantagem))
print(peca)
```

```
Vantagem apurada: R$ 4500
```

`str(vantagem)` devolve `"4500"` para a substituição. A variável `vantagem` continua guardando o número 4500.

Preencher dois campos e preservar o terceiro

```
modelo = """Investigado: <NOME>
Vantagem apurada: R$ <VALOR>
Assinatura: <ASSINATURA>"""
caso = {"investigado": "Marina Alves", "vantagem": 4500}
peca = modelo.replace("<NOME>", caso["investigado"])
peca = peca.replace("<VALOR>", str(caso["vantagem"]))
print(peca)
```

```
Investigado: Marina Alves
Vantagem apurada: R$ 4500
Assinatura: <ASSINATURA>
```

A segunda substituição usa o resultado da primeira. O campo de assinatura permanece para quem assina a peça.

Duas minutas do mesmo modelo

```
modelo = "Investigado: <NOME>"
nomes = ["Marina Alves", "Otávio Bastos"]
pecas = []
for nome in nomes:
    modelo = modelo.replace("<NOME>", nome)
    pecas.append(modelo)
print(pecas)
```

Preveja as duas minutas. A segunda ainda encontra o marcador?

Duas minutas do mesmo modelo

RESPOSTA

O que modelo guarda em cada volta

```
                              início                          fim
volta 1, nome = "Marina Alves"   Investigado: <NOME>    ->  Investigado: Marina Alves
volta 2, nome = "Otávio Bastos"  Investigado: Marina    ->  Investigado: Marina Alves
```

```
['Investigado: Marina Alves', 'Investigado: Marina Alves']
```

Na volta 2 o marcador já não existe. `.replace()` não o encontra, devolve o texto sem mudança e não acusa erro.

Preservar o modelo original

RESPOSTA

Defeito: a volta escreve por cima do modelo

```
modelo = modelo.replace("<NOME>", nome)
```

Correção: o modelo fica intacto e cada minuta é um texto novo

```
peca = modelo.replace("<NOME>", nome)
pecas.append(peca)
```

Toda volta parte do `modelo`, que conserva os marcadores. Dentro de uma mesma minuta, as substituições seguintes partem de `peca`.

A minuta de cada caso

```
modelo = "<NOME> | autos <AUTOS> | R$ <VALOR>"
casos = [
    {"investigado": "Marina Alves",
     "autos": "0000123-27.2024.8.99.0001", "vantagem": 4500},
    {"investigado": "Otávio Bastos",
     "autos": "0000456-76.2024.8.99.0001", "vantagem": 1200},
]
pecas = []
for caso in casos:
    peca = modelo.replace("<NOME>", caso["investigado"])
    peca = peca.replace("<AUTOS>", caso["autos"])
    peca = peca.replace("<VALOR>", str(caso["vantagem"]))
    pecas.append(peca)
print(pecas)
```

Que lista será mostrada? Se invertermos os casos, o que muda?

A minuta de cada caso

RESPOSTA

```
['Marina Alves | autos 0000123-27.2024.8.99.0001 | R$ 4500',
 'Otávio Bastos | autos 0000456-76.2024.8.99.0001 | R$ 1200']
```

Inverter os casos inverte a ordem das peças. Nome, autos e valor continuam juntos porque vêm do mesmo dicionário em cada volta.

No PrairieLearn: Desafio IV

Insper